

ALUMNEX CONNECT

COMPREHENSIVE FULL-STACK SOFTWARE ENGINEERING PROJECT REPORT

Evidence-based architecture, implementation, resilience, security, testing and deployment
dossier

PROJECT IDENTITY

Student / alumni / institution networking platform with mentorship, jobs, real-time communication, developer intelligence, AI career tooling and a Capacitor Android wrapper.

Prepared by	Sriman Kumar V
Department	Computer Science and Engineering
Academic Year	2026
Repository	github.com/SrimanKumarV/FSD
Primary branch	master
Report date	11 September 2026

This report is grounded in the repository source and the supplied draft report. Where implementation status cannot be conclusively established, the report explicitly labels it instead of treating documentation or dependency presence as proof.

Executive Summary

Alumnex Connect is a multi-surface full-stack platform intended to connect students, alumni, institutions and supporting ecosystem participants through professional networking, mentorship, job opportunities, events, discussion forums, real-time communication, developer-profile aggregation and AI-assisted career workflows. The implementation is organized as a React single-page application backed by Node.js/Express APIs, MongoDB/Mongoose persistence, Redis-backed capabilities, Socket.IO real-time communication, external service integrations and a Capacitor Android wrapper.

The engineering characteristics that stand out in the repository are not limited to feature count. The system contains multiple layers of defensive behavior: application-level backend failover in the Axios client, a 401 refresh queue that prevents concurrent requests from triggering competing refresh flows, CSRF handling for cookie-based requests, a mobile bearer-token path, granular authorization middleware, Redis degradation behavior, AI provider failover from Groq to Gemini, developer-stat fallback from GitHub GraphQL to REST, and user-facing network/server error messaging. These mechanisms are treated in this report as implementation evidence rather than as generic claims.

Evidence posture

Repository code is treated as the source of truth. Feature statements are classified as verified, partially implemented, configuration-dependent, mock/demo, documented/planned, or inferred. The report deliberately avoids presenting a library, comment, TODO, or planning document as proof of a completed feature.

Document Map

Section	Contents
1	Introduction and project rationale
2	Requirements and user roles
3	System architecture and data flow
4	Technology stack and engineering rationale
5	Frontend implementation dossier
6	Backend implementation dossier
7	Database design and data lifecycle
8	Authentication and authorization
9	Security architecture and threat controls
10	Module-wise functional implementation
11	AI architecture and inference workflows

Section	Contents
12	Real-time communication and presence
13	External integrations and developer intelligence
14	Caching, resilience and fallback engineering
15	Performance and optimization
16	Testing and quality assurance
17	CI/CD, deployment and operations
18	Mobile application architecture
19	Implementation status, limitations and technical debt
20	Future enhancements and conclusion
Appendices	Evidence matrices, API inventory, model inventory, environment map, test and workflow inventories

1. Introduction and Project Rationale

1.1 Problem Context

Traditional alumni directories are primarily identity stores. They often do not provide an integrated workflow for discovering mentors, requesting guidance, communicating in real time, finding jobs, showcasing projects, tracking developer activity, and obtaining structured career assistance. Alumnex Connect addresses that fragmentation by combining professional-network functions with operational workflows.

1.2 Project Objectives

- Provide a role-aware platform for students, alumni, administrators and college-oriented users.
- Support mentorship discovery, requests, sessions, reviews and mentor-oriented metadata.
- Provide career opportunities, job applications and AI-assisted job/resume workflows.
- Enable discussion, networking, real-time messaging and notifications.
- Aggregate public developer activity from multiple external platforms into a unified profile.
- Provide layered security, external-service fallbacks and operational resilience.
- Reuse the web application inside an Android wrapper rather than maintaining a completely separate native feature set.

1.3 Architectural Principles Observed

Principle	Evidence in implementation	Practical effect
Separation of concerns	Routes, middleware, services, models and utilities are separated	Business behavior is not concentrated entirely in one controller file.
Defense in depth	Helmet, CORS, rate limits, CSRF, sanitization, HPP, auth and RBAC	A request crosses multiple defensive boundaries before business logic.
Graceful degradation	AI, Redis, backend URL and developer-stat fallbacks	Selected failures can degrade functionality instead of immediately terminating user workflows.
Asynchronous I/O	Axios, MongoDB, Redis and Socket.IO in Node.js	The runtime is suited to I/O-heavy aggregation and live communication.
Reusable client state	React contexts and React Query	Authentication, profile, socket and notification state are shared across routes.

Evidence S1: backend/server.js - global server composition and middleware order

Evidence S2: frontend/src/App.js - frontend providers, lazy routes, retry and offline behavior

Evidence S3: backend/package.json / frontend/package.json - runtime and dependency inventory

2. Requirements, User Roles and System Boundaries

2.1 Functional Capability Map

Capability	Observed implementation surface	Status
Authentication	Local, Google, GitHub, email verification, password reset, 2FA, token refresh and logout	Verified
Profiles and networking	User profiles, connections/followers, developer links, social links	Verified in model/routes; extent varies by subfeature
Mentorship	Mentorship requests, mentor-oriented profile fields, sessions/reviews/rewards	Verified components exist; status of each workflow is source-dependent
Jobs and opportunities	Job posting, applications, opportunities and AI-assisted matching	Verified route/model surface; individual subfeatures vary
Events	Event management and RSVP-oriented domain	Verified route/model surface
Forum	Posts, comments, voting/reporting/moderation patterns	Verified route/model surface
Messaging	Socket.IO chat, groups and persistence	Verified
Notifications	Notification domain plus real-time delivery hooks	Verified
AI	Career chat, resume analysis, request drafting, job matching, mock interviews, interview evaluation	Verified routes with real/configuration/mock paths
Developer intelligence	External platform aggregation and normalization	Verified utilities; external dependency behavior is variable
Mobile	Capacitor Android wrapper and automated APK build	Verified

2.2 User Role Model

Role / state	Primary responsibility	Relevant controls
Student	Career discovery, mentorship requests, jobs, community participation	student / active / verified checks depending on operation
Alumni	Mentoring, job/event creation, community and professional contribution	alumni / active / approved / verified checks depending on operation
Admin	Moderation and administrative actions	admin role gate
College	Institution-oriented identity and metadata	role is present in User schema; feature depth depends on route implementation

Role / state	Primary responsibility	Relevant controls
Verified account state	Eligibility for features requiring verified identity	`verified` middleware
Approved account state	Additional trust boundary for alumni-oriented actions	`approved` and `canBeMentor` logic

2.3 System Boundaries

Inside the application boundary

React frontend, Node/Express backend, Mongoose data layer, Redis wrapper, Socket.IO handlers, authentication middleware, domain routes, services and local business logic.

Outside the application boundary

Google/GitHub OAuth, AI providers, Cloudinary, email/SMS providers, developer platforms, external job or data sources, hosting platforms and mobile platform infrastructure.

3. System Architecture and Data Flow

3.1 High-Level Architecture

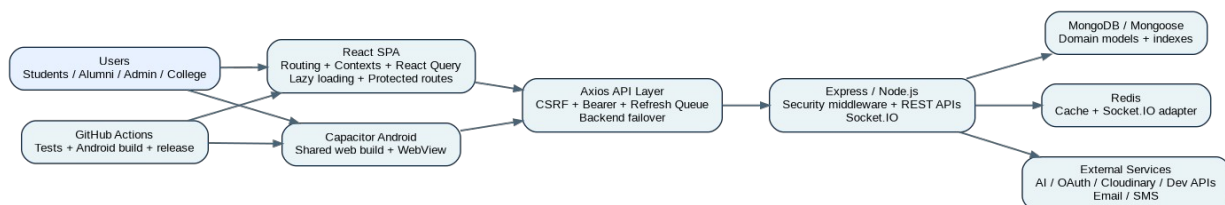


Figure 3.1 - High-level Alumnex Connect architecture

The application follows a decoupled client-server model. Browser users interact with a React SPA; the same web build is also packaged through Capacitor for Android. The client communicates with the Node/Express server through HTTP APIs and Socket.IO. The backend persists domain data in MongoDB, uses Redis where configured for caching/pub/sub behavior, and communicates with external providers for authentication, AI, file delivery and developer-data aggregation.

3.2 Request Lifecycle

Observed request-processing layers

Client request

- > CORS / security headers
- > rate limiting
- > body + cookie parsing
- > CSRF processing
- > NoSQL sanitization + HPP
- > authentication (where protected)
- > authorization / ownership

- > route validation
- > service / business logic
- > MongoDB / Redis / external API
- > response or centralized error handling

The key architectural characteristic is that the application does not rely on a single “security check”. Request acceptance is layered. Likewise, availability is not delegated to one mechanism: the frontend can switch API endpoints, AI requests can switch provider, and some local Redis-dependent behavior can degrade into a mock cache implementation.

3.3 Resilience Architecture

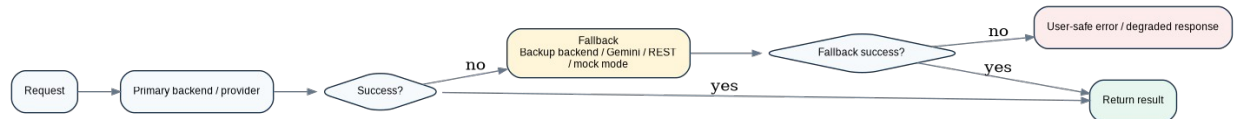


Figure 3.2 - Generic failure/fallback decision pattern observed across the system

The most important distinction is between independent fallback domains. Backend endpoint failover is a client-side mechanism. AI failover is service-level provider orchestration. Redis fallback is a runtime capability choice. GitHub developer-stat fallback is provider-specific. These should not be described as one generic load balancer; they are separate mechanisms that reduce single points of failure at different layers.

Evidence S4: frontend/src/utls/api.js - backend URL selection, failover and 401 refresh queue

Evidence S5: backend/utls/aiHelper.js - Groq to Gemini provider failover

Evidence S6: backend/utls/cache.js - real Redis versus mock/degraded cache behavior

4. Technology Stack and Engineering Rationale

Technology	Role	Observed use
React 18	Frontend application framework	Componentized SPA; lazy-loaded route pages; context providers
React Router	Client routing	Public/protected route structure
React Query	Server-state cache and retry policy	Two-hour stale time, three-hour cache, bounded retries, exponential backoff
Axios	HTTP client	Base URL failover, token handling, CSRF headers, refresh flow
Node.js + Express	Backend runtime and HTTP API	I/O-heavy REST and real-time integration
MongoDB + Mongoose	Primary persistence	Flexible domain models, validation, indexes and model hooks
Redis	Cache + Socket.IO adapter	Caching and cross-instance event propagation when configured
Socket.IO	Real-time transport	Chat, presence, notifications and reconnect/failover behavior

Technology	Role	Observed use
Cloudinary	Managed media delivery	External media storage/CDN path
bcryptjs	Password hashing	Salted password hashing before persistence
JWT	Session/authentication token	Bearer and cookie-based authentication paths
Capacitor	Mobile wrapper	Reuse React build within Android WebView
GitHub Actions	Automation	Testing and APK build/release
Jest / Supertest / in-memory MongoDB	Automated backend testing	Endpoint/service behavior with isolated test data

Why the stack fits the workload

The architecture is I/O-heavy rather than compute-heavy: many workflows combine database access, third-party requests, notifications and real-time events. Node.js, Express, Axios, MongoDB and Socket.IO are consequently coherent choices for an application whose dominant latency is external/service interaction rather than local CPU computation.

5. Frontend Implementation Dossier

5.1 Application Composition

The root application composes QueryClientProvider, ThemeProvider, AuthProvider, ProfileProvider, SocketProvider, NotificationProvider, Router and CallProvider. This makes the application state model explicit: server-state caching is handled separately from identity, profile, socket, notification and call state.

Provider hierarchy from frontend App.js

```

QueryClientProvider
  ThemeProvider
    AuthProvider
      ProfileProvider
        SocketProvider
          NotificationProvider
            Router
              CallProvider
                Layout + Routes + Overlays

```

Evidence F1: frontend/src/App.js - provider stack and route composition

5.2 Routing and Feature Surface

Path	Role / purpose
/	Public home

Path	Role / purpose
/login	Public login
/register	Public registration
/forgot-password	Password reset start
/verify-email	Email verification
/verify-2fa	2FA verification
/dashboard	Protected dashboard
/profile	Protected profile
/users/:id	Protected user profile
/network	Protected networking
/devpulse	Protected developer intelligence
/leaderboard	Protected leaderboard
/notifications	Protected notifications
/mentorship	Protected mentorship
/jobs	Protected jobs
/events	Protected events
/forum	Protected forum
/projects	Protected project showcase
/tech-hub	Protected tech hub
/career-board	Protected opportunity board
/project-collaboration	Protected collaboration
/resume	Protected resume analyzer
/businesses	Protected business directory
/chat	Protected chat
/admin	Admin-only
/admin-dashboard	Admin-only dashboard
/privacy	Public legal
/terms	Public legal
/cookies	Public legal

Path	Role / purpose
/help	Public support
/contact	Public support
/faq	Public support

5.3 Lazy Loading and Suspense

Major pages are imported with `React.lazy` and rendered under `Suspense`. This turns route modules into asynchronously loaded chunks rather than forcing the entire feature surface into the initial bundle. The application also places the route tree under an `ErrorBoundary`, creating a separate failure boundary for rendering errors.

5.4 Server-State Policy

Policy	Observed value	Engineering implication
Retry	No retry for 401/403/404; otherwise up to 2 retries	Auth and authorization errors are not pointlessly retried
Backoff	$\min(1000 * 2^{\text{attempt}}, 10000)$	Avoids immediate repeated requests
Window focus refetch	Disabled	Reduces background refresh traffic
Stale time	2 hours	Navigation can reuse cached server state
Cache time	3 hours	Data can persist in memory beyond stale window
Mutations	No automatic retry	Avoids accidental duplicate writes

5.5 Offline Awareness

The frontend registers ``online`` and ``offline`` browser events and displays a high-priority banner when ``navigator.onLine`` becomes false. This is a user-experience safeguard rather than a complete offline application: the message explicitly warns that some features may not work.

5.6 HTTP Client Resilience

The `Axios` wrapper is one of the most technically distinctive frontend files. It detects `Capacitor`, maintains a mobile token store, constructs a candidate backend URL list, randomizes the initial backend when more than one endpoint is available, fails over on network/timeout errors, queues concurrent requests during token refresh, distinguishes auth failures from network failures, and exposes a `Promise.allSettled` helper for independent parallel requests.

Frontend resilience patterns

```
Primary backend URL
| network / timeout
v
```

```

triggerFailover()
|
+--> next backend URL
|
+--> retry original request

401 responses
|
+--> if refresh in progress: queue request
+--> else: refresh token once
+--> replay waiting requests if refresh path succeeds

```

Evidence F2: frontend/src/utils/api.js - failover, token refresh queue, CSRF token memory and parallel-request helper

6. Backend Implementation Dossier

6.1 Server Composition

The backend starts from an HTTP server wrapping Express and attaches Socket.IO. The server configures proxy awareness, dynamically assembled allowed origins, Redis adapter support, global security middleware, rate limits, parsers, CSRF protection, sanitization, route registration and centralized error handling.



Figure 6.1 - Defensive request pipeline

6.2 Security Middleware Sequence

Layer	Observed mechanism	Purpose
1	Helmet with CSP and frameguard	Browser security headers and clickjacking reduction
2	Compression	Response size reduction
3	Morgan	HTTP request logging
4	CORS	Origin/credential policy
5	General rate limit	Request volume control
6	Auth-specific rate limit	Brute-force resistance on login/register
7	JSON / URL-encoded parser	Request body decoding with 10 MB limit
8	Cookie parser	Cookie-based token and CSRF paths
9	CSRF middleware	State-changing request verification
10	Mongo sanitize	NoSQL operator injection reduction
11	HPP	Repeated-parameter pollution reduction
12	Route/auth middleware	Identity, roles and ownership

6.3 Route Domains

#	Route prefix	Domain responsibility
1	/api/auth	Logical domain router
2	/api/users	Logical domain router
3	/api/mentorship	Logical domain router
4	/api/jobs	Logical domain router
5	/api/events	Logical domain router
6	/api/forum	Logical domain router
7	/api/contests	Logical domain router
8	/api/messages	Logical domain router
9	/api/admin	Logical domain router
10	/api/notifications	Logical domain router
11	/api/upload	Logical domain router
12	/api/dev-activity	Logical domain router
13	/api/leaderboard	Logical domain router
14	/api/feedback	Logical domain router
15	/api/helpdesk	Logical domain router
16	/api/projects	Logical domain router
17	/api/institutions	Logical domain router
18	/api/tasks	Logical domain router
19	/api/ai	Logical domain router
20	/api/tech-hub	Logical domain router
21	/api/business	Logical domain router

6.4 Error and Process-Level Safety

The server has a centralized error handler that logs the error stack and deliberately avoids exposing internal stack details in production. A bottom-of-stack wildcard handler returns a 404 JSON response. Process-level listeners log unhandled promise rejections and uncaught exceptions. These are not substitutes for graceful shutdown or process supervision, but they demonstrate deliberate attention to failure visibility and information disclosure.

Evidence B1: backend/server.js - error handler, 404 handler, process listeners

7. Database Design and Data Lifecycle

7.1 Model Inventory

#	Model	Primary responsibility
1	User	Domain entity
2	Message	Domain entity
3	ChatGroup	Domain entity
4	Contest	Domain entity
5	DevProfile	Domain entity
6	Event	Domain entity
7	Feedback	Domain entity
8	ForumPost	Domain entity
9	HelpDesk	Domain entity
10	Job	Domain entity
11	JobApplication	Domain entity
12	JobOpportunity	Domain entity
13	MentorReview	Domain entity
14	MentorReward	Domain entity
15	Mentorship	Domain entity
16	MentorshipRequest	Domain entity
17	MentorshipSession	Domain entity
18	Notification	Domain entity
19	Project	Domain entity
20	Report	Domain entity
21	Task	Domain entity
22	TechHub	Domain entity
23	Business	Domain entity

7.2 User Model - Detailed Evidence

Area	Fields	Implementation detail
Identity	name, email, phoneNumber, phoneVerified	Name is required; email unique and normalized; password is hidden from default selection
Role	role	Enum: student, alumni, admin, college
Profile	photo, bio, skills, interests, location, country, college, department	Trimmed profile fields and bounded bio
Student domain	studentInfo.course/year/university	Structured student metadata
Alumni domain	graduationYear, company, position, industry, experience, mentor metadata	Mentor capacity/rating/availability fields
College domain	collegeInfo.*	Institution extraction and metadata surface
Social	linkedin, github, twitter, website, portfolio, codeforces, leetcode, codolio	External profile references
Status	status, isVerified, isApproved, isActive	Multiple account-state gates
Network	connections, followers, following, followRequests	Self-referencing User ObjectIds
Activity	lastActive	Used for activity/online calculations
Security tokens	resetPasswordOtp, verificationOtp, deleteAccountOtp, twoFactorOtp + expiries	Temporary authentication/verification state
Timestamps	createdAt, updatedAt	Mongoose timestamps

7.3 User Model Behaviors

- Password hashing uses bcrypt with a generated salt of 12 rounds before save when the password is modified.
- `comparePassword()` delegates candidate verification to bcrypt.
- `updateLastActive()` performs a targeted update rather than requiring the whole document to be saved synchronously.
- `getPublicProfile()` removes `password` and `__v` from the returned object.
- `isOnline` is computed from activity within a five-minute window.
- Indexes exist for alumni company, skills, role/status, alumni industry and location.
- A document delete hook removes related forum posts, messages, mentorships, notifications and jobs and cleans relationship arrays in other users.

7.4 Data Integrity and Lifecycle

Cascade behavior

The User model attempts an explicit cascade-like cleanup across related collections when a user document is deleted. This is important in a document database because references are not automatically cascaded by

MongoDB in the way foreign keys may cascade in a relational system.

Evidence D1: backend/models/User.js - validation, indexes, password hashing, virtuals and delete cleanup

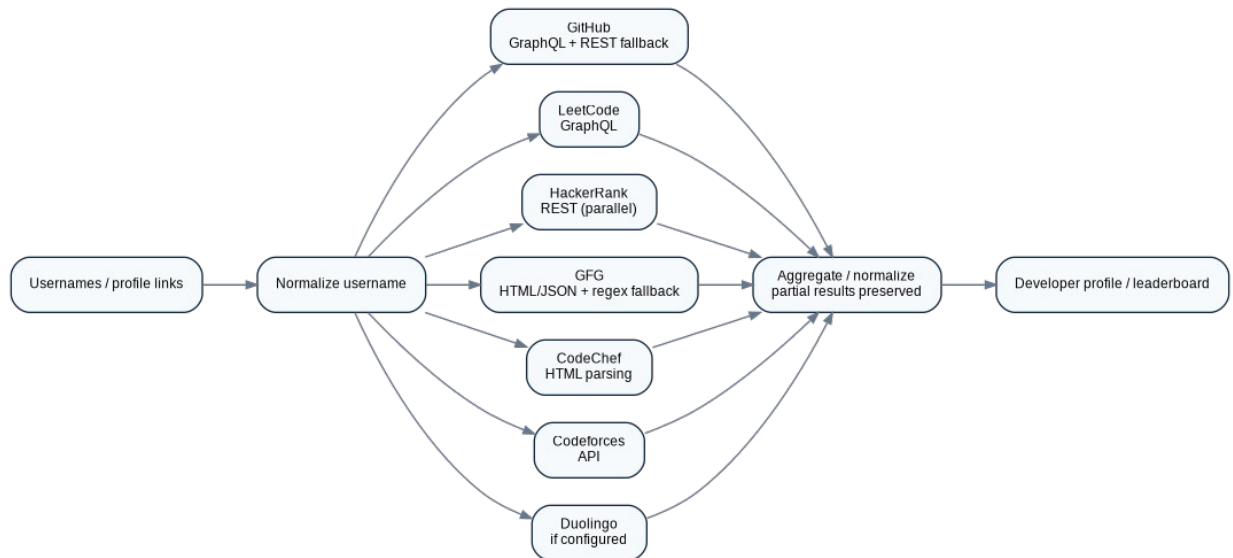


Figure 7.1 - Developer-data aggregation path (external to Mongo persistence)

8. Authentication and Authorization

8.1 Authentication Sources

Mechanism	Observed implementation	Security boundary
Local credentials	Email + bcrypt password	Password verification + JWT
Google OAuth	Dedicated endpoint	Provider identity verification
GitHub OAuth	Dedicated endpoint + mobile callback paths	Authorization-code exchange path
Email verification	OTP endpoint	Verified account state
Password reset	Email + OTP	Temporary reset token/OTP
2FA	Email or SMS OTP	Additional verification factor
Bearer token	Authorization header	Used by clients including mobile path
Cookie token	HttpOnly cookie	Browser-oriented token path

8.2 Cookie and Bearer Fallback

Authentication token selection

Protected request

|

```

+-- cookie token available? --> verify cookie JWT
|
+-- otherwise Authorization: Bearer <token> --> verify bearer JWT
|
+-- no token --> 401

```

The `protect` middleware first checks a cookie token and then falls back to a Bearer header. After verification it reloads the user document, excludes the password field, rejects missing users, rejects inactive accounts and asynchronously updates last activity. Role and state middleware then create additional authorization boundaries.

8.3 Authorization Layers

Control	Condition	Failure
Authenticated	valid JWT + active user	401
Admin	role === admin	403
Alumni	role === alumni	403
Student	role === student	403
Verified	isVerified === true	403
Approved	isApproved === true	403
Resource ownership	current user matches resource user id, unless admin	400/403
Mentor eligibility	alumni + active + approved	403
Mentorship requester	student + active	403
Forum poster	active + verified	403

Evidence A1: backend/middleware/auth.js - authentication, role, state and ownership middleware

Evidence A2: backend/routes/auth.js - registration, login, OAuth, reset, verification and 2FA route guards

9. Security Architecture and Threat Controls

9.1 Control Matrix

Threat	Control	Effect / limitation
Credential theft	HttpOnly JWT cookie path; Bearer token path for clients	Cookie path reduces script-level access; mobile token has different exposure characteristics
CSRF	Double-submit style cookie + header check	Production rejects missing/mismatched token; safe methods bypass
NoSQL injection	express-mongo-sanitize	Removes Mongo-style operators from

Threat	Control	Effect / limitation
		request input
HPP	hpp middleware	Reduces repeated-parameter ambiguity
Brute force	Auth-specific rate limit 10 requests / 15 min	Limits repeated login/register attempts per key
General abuse	API rate limit 5000 requests / 15 min	Broad request-volume control
Clickjacking	Helmet frameguard deny + CSP frameAncestors none	Blocks embedding in frames
CORS misuse	Allowlist in production; credentials enabled	Rejects non-allowed origins
Information disclosure	Production error response excludes stack	Internal implementation details stay server-side
Broken access control	Role + verification + approval + ownership checks	Unauthorized requests receive 401/403
Sensitive fields	Password `select:false` + public-profile stripping	Reduces accidental response exposure



```

graph LR
    Request([Request]) --> Helmet_CSP([Helmet / CSP])
    Helmet_CSP --> CORS([CORS])
    CORS --> Rate_limit([Rate limits])
    Rate_limit --> Parsers_cookies([Parsers / cookies])
    Parsers_cookies --> CSRF([CSRF])
    CSRF --> Sanitize_hpp([Sanitize + HPP])
    Sanitize_hpp --> JWT_OAuth([JWT / OAuth])
    JWT_OAuth --> Role_ownership([Role + ownership])
    Role_ownership --> Validation([Validation])
    Validation --> Business_logic([Business logic])
    Business_logic --> DB_Redis_APIs([DB / Redis / APIs])
    DB_Redis_APIs --> Safe_response([Safe response])
  
```

Figure 9.1 - Security boundary map

9.2 CSRF Behavior in Detail

The custom CSRF middleware generates a random 32-byte hexadecimal token when the XSRF cookie is absent, exposes the token through the response header, skips validation for GET/HEAD/OPTIONS, bypasses the state-changing check for the Capacitor marker, and in production returns HTTP 403 when the header is missing or mismatched. Development mode intentionally allows requests through after logging a warning. That development bypass is a configuration trade-off, not a production security feature.

Evidence SEC1: backend/middleware/csrf.js - token creation, safe methods, mobile exception and production enforcement

9.3 Residual Risks

- The mobile path stores a bearer token in localStorage. This is a conscious architecture trade-off for the wrapper environment and carries greater token-extraction risk on a compromised device than an HttpOnly cookie.
- Client-side backend failover improves availability but is not equivalent to a managed infrastructure load balancer or global traffic manager.
- The report should not assume file malware scanning, distributed rate-limiting, automated secret rotation or full observability unless additional evidence exists.
- External scraping and third-party APIs remain subject to upstream schema/rate-limit changes.

10. Module-wise Functional Implementation

Module	Purpose	Primary evidence
Authentication	Identity, verification, OAuth, 2FA, token lifecycle	auth.js, authService.js, auth middleware
Profiles / network	User metadata, connections and developer links	User model + users routes + profile pages
Mentorship	Mentor discovery/request/session/review domain	Mentorship-related models/routes
Jobs	Listings, applications and AI support	jobs routes/services + Job models
Events	Community/event lifecycle	events route + Event model
Forum	Posts/comments/voting/moderation	forum route + ForumPost model + moderation utility
Messaging	Direct/group real-time communication	messages route + Socket.IO + Message/ChatGroup models
Notifications	Persistent and real-time signals	Notification model/routes/socket handlers
Developer activity	Multi-platform stats normalization	devStatsFetcher.js + DevProfile
AI	Career chat, resume, job match, interview	ai.js + aiHelper.js
Projects	Project showcase/collaboration	Project model/routes
Helpdesk	Support tickets	HelpDesk model/routes
Business	Business directory	Business model/routes
Tasks	Task lifecycle and user completion	Task model/routes
Tech Hub	Technology resources/news domain	TechHub model/routes
Admin	Administrative controls	admin routes/pages

10.1 Mentorship

The mentorship domain includes distinct models for mentorship requests, active mentorship relationships, sessions, mentor reviews and rewards. The User model itself contains mentor readiness fields such as availability, maximum mentees, preferred batches, rating, review count and accumulated mentorship hours. The authorization layer further restricts mentor eligibility to active, approved alumni.

Mentorship domain flow - report model based on repository entities; individual transition details should be matched to the current route logic before claiming every transition as executed

```
Student
-> mentor discovery
-> request creation
-> authorization / validation
-> notification
```

```
-> mentor decision
-> mentorship lifecycle
-> session / review / reward data
```

10.2 Jobs and Resume Analysis

The job domain combines conventional listing/application workflows with AI-assisted analysis. The resume-analysis route accepts a PDF, checks MIME type, parses the file from memory using pdf-parse, constructs a structured prompt containing the job description and extracted resume text, requests strict JSON output from the AI orchestrator, strips code-fence wrappers if present, extracts a JSON object and parses it before returning structured analysis.

Resume analysis processing chain

```
PDF upload
-> multipart memory buffer
-> MIME check == application/pdf
-> pdf-parse
-> resume text
-> JD + resume prompt
-> Groq / Gemini
-> remove markdown fences
-> extract JSON object
-> JSON.parse
-> structured ATS response
```

Evidence M1: backend/routes/ai.js - resume upload, MIME validation, pdf-parse and response parsing

10.3 Forum and Moderation

The forum route imports a moderation utility, user and notification models, and the AI helper. This indicates that the discussion subsystem is not purely a CRUD post store; moderation and notification-related behavior are part of its implementation surface. Exact trigger matrices should remain tied to the route code when extending this chapter with further endpoint-level evidence.

10.4 Developer Intelligence

The developer profile fetcher normalizes usernames before hitting external services, strips leading @ signs, handles profile URLs, uses parallel requests where useful, preserves partial success through Promise.allSettled, and applies provider-specific fallbacks. The GitHub implementation prefers GraphQL for rich data and falls back to the v3 REST endpoint when GraphQL fails. GFG parsing first attempts embedded JSON and then falls back to regex extraction.

Evidence M2: backend/utils/devStatsFetcher.js - username normalization, parallel external calls, GitHub GraphQL/REST and GFG parsing fallback

11. AI Architecture and Inference Workflows

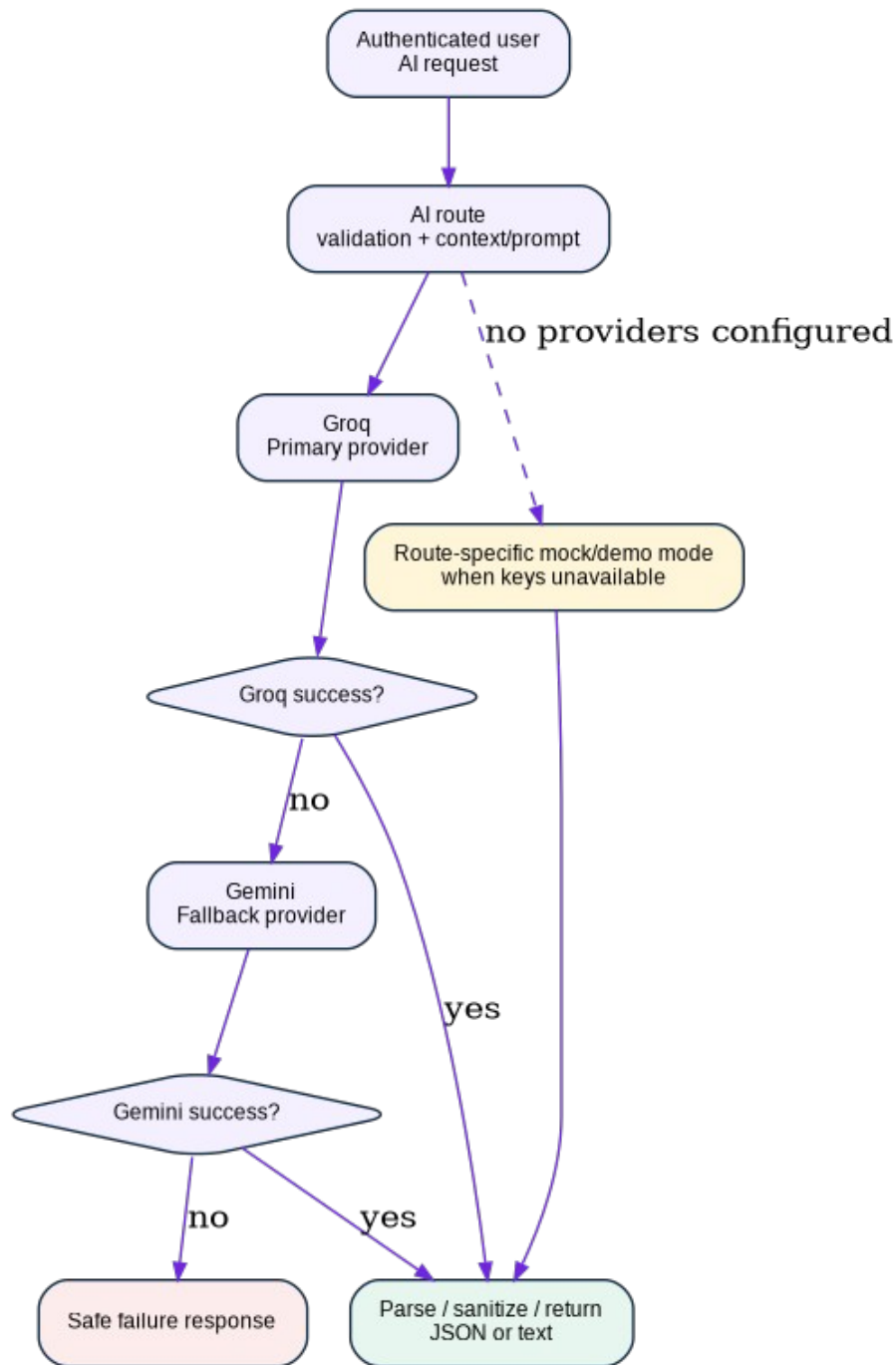


Figure 11.1 - AI provider orchestration and demo-mode behavior

11.1 Provider Orchestration

The AI helper exposes separate functions for Groq and Gemini and a coordinator named `callAIWithFallback`. The coordinator first attempts Groq using the configured compound-mini model and then catches failure before attempting Gemini. If Gemini also fails, it raises a final error indicating that both providers failed or are unconfigured.

11.2 AI Route Inventory

Endpoint	Purpose	Input surface	Output mode
POST /api/ai/chat	Career mentor chat	message + optional history	Text
POST /api/ai/analyze-resume	ATS-style resume analysis	PDF resume + job description	Structured JSON
POST /api/ai/draft-request	Mentorship networking request draft	mentor/student profile data	Text draft
POST /api/ai/match-job	Job fit evaluation	job description + skills + experience	Structured JSON
POST /api/ai/mock-interview	Interactive interviewer	message + history + role + experience	Text
POST /api/ai/evaluate-interview	Whole-session evaluation	history + role + experience	Structured JSON

11.3 Demonstration / Mock Mode

Several AI endpoints deliberately return deterministic demo output when neither Groq nor Gemini credentials are present. This is an important distinction: demo mode proves that the UI workflow can operate without a live provider, but it is not equivalent to live model inference. The final report therefore classifies this behavior as a fallback/demo mechanism rather than as AI capability itself.

11.4 Output Robustness

For JSON-oriented AI tasks, the implementation requests a JSON response mode where supported, then defensively removes markdown fences and extracts the first JSON object-shaped block before parsing. This is a lightweight normalization layer aimed at common formatting deviations from model output.

Evidence AI1: backend/utils/aiHelper.js - Groq/Gemini functions and failover coordinator

Evidence AI2: backend/routes/ai.js - route-level mock behavior, prompt construction, PDF parsing and output normalization

12. Real-Time Communication, Messaging and Presence

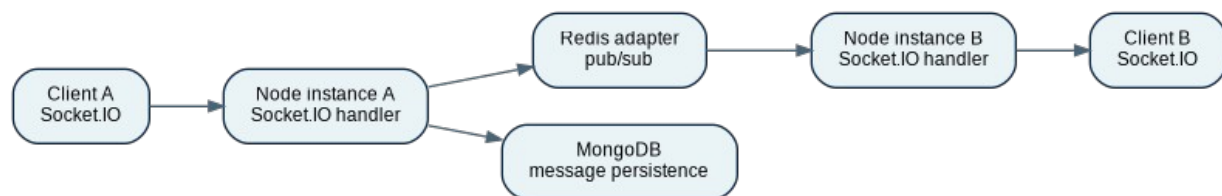


Figure 12.1 - Socket.IO with Redis adapter concept

12.1 Client Socket Lifecycle

The SocketContext creates a Socket.IO connection only when a user exists. On logout or user removal it disconnects and clears state. It tracks `isConnected`, maintains an online-user map, joins a personal user room, requests current online users, reacts to user online/offline events, and marks a user away/back based on browser visibility. Cleanup removes event listeners and disconnects the socket on unmount.

12.2 Socket Failover

Socket connection failures trigger the same backend failover mechanism used by the Axios client. If HTTP failover changes the active backend, a `backend-failover` custom event is emitted so SocketContext can swap its Socket.IO URI to match the new backend and reconnect. This is a strong cross-layer consistency mechanism: API and WebSocket traffic are intentionally kept pointed at the same active backend.

Evidence RT1: frontend/src/contexts/SocketContext.js - socket creation, presence state, failover event handling and cleanup

Evidence RT2: backend/server.js - Socket.IO initialization and Redis adapter configuration

13. External Integrations and Developer Intelligence

Service	Domain	Auth	Role	Fallback
Groq	AI	API key	Primary AI provider	Gemini
Gemini	AI	API key	AI fallback provider	Route-specific mock/degraded output where implemented
Google OAuth	Authentication	OAuth credential	Identity provider	Application error path
GitHub OAuth	Authentication	Client/application credentials	Identity provider	Application error path
GitHub GraphQL	Developer stats	Optional token	Rich developer data	GitHub REST
GitHub REST	Developer stats	Optional token	Fallback developer data	Null / error path
LeetCode GraphQL	Developer stats	Public endpoint	Problem/contest data	Null result on failure
HackerRank REST	Developer stats	Public endpoint	Badges/certificates/profile	Null result on profile failure
GFG web parsing	Developer stats	HTTP request	Coding score / solved data	Regex fallback
CodeChef web parsing	Developer stats	HTTP request	Rating/profile data	Error/partial path
Codeforces API	Developer stats	Public endpoint	Competitive programming metrics	Error/partial path
Cloudinary	Media	API credentials	Managed upload/delivery	Upload error
Email/SMS providers	Verification/notifications	Provider credentials	OTP and notifications	Route-specific error handling
Judge0	Contest execution	Configured API	Code evaluation	Classify by actual route implementation status

External services are inherently outside the application's control boundary. The most resilient integration points therefore normalize inputs, preserve partial results where possible, and avoid treating one external provider as the only source of truth. Developer-stat aggregation is especially explicit in this respect: the fetcher returns null/partial results for failed modules rather than requiring every platform to succeed.

Evidence I1: backend/utils/devStatsFetcher.js - developer data integrations and provider-specific fallbacks

14. Caching, Resilience and Fallback Engineering

14.1 Redis Runtime Modes

The cache utility has two implementations: a real Redis client and a mock cache. Real Redis is selected in production or when REDIS_URL is present. Otherwise a mock client exposes the same method names expected by dependent code. The real client serializes values, applies an EX TTL, and catches get/set/delete errors so cache failures can degrade into cache misses or falsey writes rather than directly crashing application flows.

Failure / condition	Observed behavior	Engineering effect
No REDIS_URL outside production	Mock cache selected	Local development can run without a Redis server
Redis get fails	Undefined returned	Caller can continue as cache miss
Redis set fails	False returned	Persistence to cache is non-blocking to core flow
Redis delete fails	0 returned	Caller receives safe no-op-like result
Redis adapter connection fails	Error logged	Main server path continues with degraded socket scaling
AI provider failure	Provider-level fallback	Service continuity
Backend timeout	API URL failover	Request can be retried on another configured backend
Concurrent 401s	Refresh queue	One refresh path rather than many simultaneous attempts

Evidence R1: backend/utils/cache.js - Redis mode selection and safe cache wrappers

Evidence R2: frontend/src/utils/api.js - backend failover and token refresh queue

14.2 Failover Register

Subsystem	Primary	Detection	Fallback	Recovery	User impact
HTTP API	Primary backend	network/timeout	Next backend URL	Axios request retry	User may not notice if backup is healthy
Socket.IO	Current backend	connect_error	Next backend Socket.IO endpoint	Reconnect after 1 second	Live connection briefly interrupted
JWT refresh	Existing token	401	single refresh request + queued callers	Replay original requests	Logout only on explicit auth failure
AI	Groq	exception	Gemini	same AI route	Usually transparent
AI no providers	Missing keys	environment inspection	mock/demo response for supported endpoint	No external inference	Feature remains demonstrable

Subsystem	Primary	Detection	Fallback	Recovery	User impact
GitHub developer stats	GraphQL	API error / errors payload	GitHub REST	re-fetch user data	Reduced richness
GFG parsing	Embedded JSON	parse failure / no values	regex parsing	extract score/solved values	Reduced parsing robustness
Redis	Real client	connection/read/write exception	safe miss/no-op mock/degraded path	Continue core request	Caching/scale features degrade

15. Performance and Optimization

Optimization	Implementation	Expected benefit
Frontend code splitting	React.lazy for page-level modules	Smaller initial bundle / deferred route cost
Server-state caching	React Query staleTime 2h / cacheTime 3h	Fewer duplicate reads
Retry backoff	Exponential delay up to 10s	Avoids tight retry loops
Mutation policy	No automatic mutation retries	Prevents duplicate writes
Request-level timeout	Axios 60s	Accounts for slow/waking backend while bounding hang time
Parallel external calls	Promise.allSettled	Reduces overall aggregation latency and preserves partial success
Compression	Express compression	Smaller HTTP responses
Database indexes	User indexes on common filters	Faster common queries
Fire-and-forget activity update	Last-active update without blocking protected request	Removes non-critical write latency from request path
Redis TTL	EX 1800 default in cache wrapper	Bounds cache lifetime
Socket.IO Redis adapter	Pub/sub across instances	Enables horizontal event propagation when deployed multi-instance
Cloudinary	Managed media delivery	Offloads media storage/delivery from app server

15.1 Important Trade-offs

- Long React Query stale windows improve responsiveness but increase the age of data reused from cache.
- A 60-second Axios timeout reduces false failure during cold-start recovery but can make a genuinely unavailable request feel slow.
- Client-side endpoint failover improves resilience but retains endpoint-selection state inside the client process rather than an infrastructure control plane.
- In-memory PDF handling is simple, but memory usage scales with upload size and concurrency; production-grade systems may move this work to a queue or object storage pipeline.

16. Testing and Quality Assurance

16.1 Automated Test Toolchain

Tool	Purpose	Observed role
Jest	Unit/service/API test execution	Backend test runner
Supertest	HTTP integration testing	Exercises Express routes without browser
mongodb-memory-server	Database isolation	Supports tests without using production MongoDB
React Testing Library	Component testing	Frontend interaction tests
Cypress	End-to-end/browser automation	Available in frontend toolchain
GitHub Actions	Continuous test execution	Backend test workflow

16.2 CI Test Workflow

Observed backend test workflow

```

Push / pull request
-> checkout
-> Node 18 setup + npm cache
-> npm ci in backend
-> npm test
  
```

Evidence T1: `.github/workflows/test.yml` - Node.js 18, npm ci and npm test

The repository contains tests named for authentication, authentication services, job services, mentorship services and RBAC in the supplied draft report; the exact breadth and current pass rate should be reported from the current test run rather than inferred from filenames alone.

17. CI/CD, Deployment and Operations

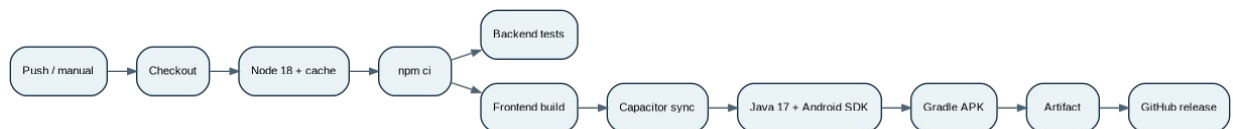


Figure 17.1 - Android build and release workflow

17.1 Verified Workflow Surface

Workflow	Trigger	Observed responsibilities
test.yml	push / pull request to main (per workflow)	Backend dependency install and Jest test execution
build-apk.yml	push to main/master + manual dispatch	Frontend build, Capacitor sync, Android toolchain, debug APK, artifact and GitHub

Workflow	Trigger	Observed responsibilities
		release
production-monitor.yml	Repository workflow present	Operational monitoring intent; exact current semantics should be taken from the workflow source before claiming specific checks

17.2 Android Build Pipeline

The Android workflow checks out the repository, installs the frontend, runs the frontend build with explicit environment values, installs the mobile wrapper, copies the React build into `www`, runs Capacitor synchronization, configures Java 17 and Android tooling, makes `gradlew` executable, builds a debug APK, uploads the APK artifact, and creates a GitHub release carrying the generated APK.

Evidence C1: `.github/workflows/build-apk.yml` - complete Android build/release chain

17.3 Deployment Classification

Current-vs-possible deployment rule

A technical report must distinguish actual deployment evidence from documented hosting recommendations. The repository documentation discusses Render/Vercel/other providers, but a report should not claim “currently hosted on X” unless deployment evidence or configuration proves it.

18. Mobile Application Architecture

The mobile application is a Capacitor wrapper around the web build rather than a separate native Kotlin implementation. The Capacitor configuration declares appId `com.alumnex.connect`, app name `Alumnex Connect`, a `www` web directory, HTTPS Android scheme and cleartext allowance. The Android CI flow synchronizes the compiled React assets into this wrapper before Gradle builds the APK.

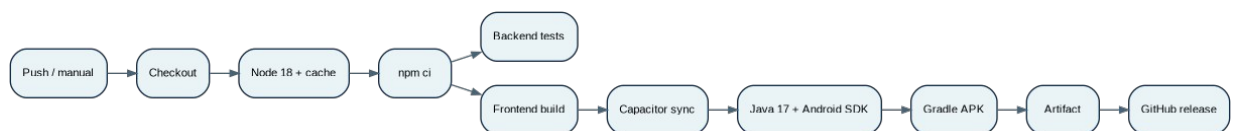


Figure 18.1 - Reuse of the React web build inside Capacitor/Android

18.1 Mobile Authentication Trade-off

The browser and mobile paths do not share identical cookie assumptions. The frontend API utility detects Capacitor environments and keeps a JWT in localStorage, then adds an Authorization Bearer header and the mobile marker used by the backend CSRF middleware. This makes the mobile path operational in a WebView environment where cross-origin cookie behavior can be unreliable, but it changes the token-exposure model compared with HttpOnly browser cookies.

Evidence MO1: `mobile-wrapper/capacitor.config.json` - Capacitor app identity and web directory

Evidence MO2: `frontend/src/utis/api.js` - Capacitor detection and mobile token store

Evidence MO3: `backend/middleware/csrf.js` - mobile bearer-token path and CSRF exception

19. Implementation Status, Limitations and Technical Debt

19.1 Evidence Classification Matrix

Capability	Classification	Evidence basis
MERN foundation	Verified	React frontend + Express/Node backend + Mongo/Mongoose
Granular authorization	Verified	Multiple dedicated middleware functions
AI provider failover	Verified	Groq -> Gemini coordinator
AI demo mode	Verified	Routes return explicit mock data when providers are unconfigured
Redis fallback wrapper	Verified	Mock/real selection
Client backend failover	Verified	Axios active URL rotation
Token refresh queue	Verified	Concurrent 401 requests are queued
Socket failover synchronization	Verified	Custom backend-failover event updates socket endpoint
Developer-stat fallback	Verified for observed services	GitHub GraphQL -> REST; GFG parsing fallback
Android automated build	Verified	GitHub Actions workflow builds APK
Production infrastructure load balancer	Not established from source alone	Client-side failover exists; infrastructure LB is a separate claim
Automatic secret rotation	Not established	Treat as future enhancement unless additional evidence exists
Full malware scanning for uploaded files	Not established	MIME validation/PDF parsing is not equivalent to AV scanning
Full distributed tracing	Not established	Recommend separately

19.2 Technical Debt / Hardening Opportunities

- Move heavy PDF/AI workloads to background workers or queues at scale.
- Add formal schema/version migration strategy if model evolution becomes frequent.
- Introduce centralized observability and distributed tracing for cross-service flows.
- Consider distributed rate-limiting storage when scaling across multiple backend instances.
- Add stronger file-content scanning if user uploads become security-critical.
- Formalize API contract documentation (OpenAPI) from actual routes and validators.
- Measure performance empirically with load tests rather than relying only on structural optimizations.
- Review third-party scraping dependence and add source freshness/contract monitoring.

20. Future Enhancements and Conclusion

20.1 Recommended Evolution Path

Phase	Enhancement direction
Near term	Formal API specification, stronger automated E2E tests, observability, deployment-state documentation
Scale phase	Queue-based AI/PDF work, distributed cache/rate-limit strategy, stronger background processing
Security hardening	Secret rotation, file scanning, security regression suite, audit logging
Data engineering	Provider freshness monitoring, normalized external-data schemas, background aggregation
Operations	Metrics, traces, alerting, backup/restore drills and disaster-recovery procedures

20.2 Conclusion

Alumnex Connect is best understood as a multi-domain full-stack platform rather than a single-feature alumni directory. Its implementation includes a React application with substantial route-level code splitting and server-state policies, a Node/Express backend with layered security and authorization, MongoDB/Mongoose models with indexes and lifecycle hooks, Socket.IO real-time capabilities with Redis adapter support, AI provider orchestration with explicit fallback paths, external developer-data aggregation, managed media integration, and a Capacitor-based Android delivery path. The strongest evidence of engineering depth is the set of cross-cutting mechanisms that address failure, not simply the number of pages or libraries.

For academic/evidence purposes, the most defensible claims are the ones tied directly to source files and functions. This report therefore treats the repository as the implementation record and explicitly separates verified mechanisms from documentation-only or configuration-dependent capabilities. That approach makes the report useful not only as a project description but as a traceable record of the engineering work present in the codebase.

Appendix A - Evidence Source Register

ID	Source	Purpose
S1	backend/server.js	Express/Socket.IO server composition and global middleware
S2	frontend/src/App.js	Provider hierarchy, routes, QueryClient policy and offline banner
S3	backend/package.json	Backend dependencies and scripts
S4	frontend/src/utlis/api.js	Backend URL failover, token refresh queue, mobile bearer path and parallel request helper
S5	backend/utlis/aiHelper.js	Groq primary + Gemini fallback orchestration
S6	backend/utlis/cache.js	Real Redis and mock/degraded cache implementations
S7	backend/middleware/auth.js	JWT protection, role gates, verification, approval and ownership checks
S8	backend/middleware/csrf.js	Double-submit-style CSRF behavior and mobile exception
S9	backend/models/User.js	User schema, indexes, password hashing, virtuals and delete cleanup
S10	backend/routes/ai.js	AI endpoint implementations, PDF parsing, JSON normalization and mock behavior
S11	frontend/src/contextes/SocketContext.js	Socket lifecycle, presence and failover synchronization
S12	backend/utlis/devStatsFetcher.js	Developer-stat integrations and provider-specific fallbacks
S13	.github/workflows/test.yml	Backend CI test workflow
S14	.github/workflows/build-apk.yml	Android build and release automation
S15	mobile-wrapper/capacitor.config.json	Capacitor application configuration

Appendix B - Selected API / Domain Inventory

Domain	Route prefix	Representative responsibilities
auth	/api/auth	Domain-specific REST operations
users	/api/users	Domain-specific REST operations
mentorship	/api/mentorship	Domain-specific REST operations

Domain	Route prefix	Representative responsibilities
jobs	/api/jobs	Domain-specific REST operations
events	/api/events	Domain-specific REST operations
forum	/api/forum	Domain-specific REST operations
contests	/api/contests	Domain-specific REST operations
messages	/api/messages	Domain-specific REST operations
admin	/api/admin	Domain-specific REST operations
notifications	/api/notifications	Domain-specific REST operations
upload	/api/upload	Domain-specific REST operations
dev-activity	/api/dev-activity	Domain-specific REST operations
leaderboard	/api/leaderboard	Domain-specific REST operations
feedback	/api/feedback	Domain-specific REST operations
helpdesk	/api/helpdesk	Domain-specific REST operations
projects	/api/projects	Domain-specific REST operations
institutions	/api/institutions	Domain-specific REST operations
tasks	/api/tasks	Domain-specific REST operations
ai	/api/ai	Domain-specific REST operations
tech-hub	/api/tech-hub	Domain-specific REST operations
business	/api/business	Domain-specific REST operations

Appendix C - Database Model Inventory

#	Model	Domain
1	User	Alumnex Connect domain
2	Message	Alumnex Connect domain
3	ChatGroup	Alumnex Connect domain
4	Contest	Alumnex Connect domain
5	DevProfile	Alumnex Connect domain

#	Model	Domain
6	Event	Alumnex Connect domain
7	Feedback	Alumnex Connect domain
8	ForumPost	Alumnex Connect domain
9	HelpDesk	Alumnex Connect domain
10	Job	Alumnex Connect domain
11	JobApplication	Alumnex Connect domain
12	JobOpportunity	Alumnex Connect domain
13	MentorReview	Alumnex Connect domain
14	MentorReward	Alumnex Connect domain
15	Mentorship	Alumnex Connect domain
16	MentorshipRequest	Alumnex Connect domain
17	MentorshipSession	Alumnex Connect domain
18	Notification	Alumnex Connect domain
19	Project	Alumnex Connect domain
20	Report	Alumnex Connect domain
21	Task	Alumnex Connect domain
22	TechHub	Alumnex Connect domain
23	Business	Alumnex Connect domain

Appendix D - Environment / Configuration Map

Variable / setting	Purpose	Observed behavior
MONGODB_URI	MongoDB connection string	Falls back to local Mongo URI in server.js
JWT_SECRET	JWT signing secret	Required by authentication logic
REDIS_URL	Redis connection	Selects real Redis client when set
NODE_ENV	Environment behavior	Controls test/production security and service initialization
FRONTEND_URL	Allowed production origin(s)	Comma-separated additional CORS origins
GROQ_API_KEY	Primary AI credentials	Absence drives provider fallback / route

Variable / setting	Purpose	Observed behavior
		demo mode
GEMINI_API_KEY	Fallback AI credentials	Used when Groq fails or for demo absence logic
REACT_APP_API_URL	Primary frontend API	Used in production candidate backend list
REACT_APP_BACKUP_API_URL	Backup frontend API	Used for client-side failover
REACT_APP_BACKUP_API_URL_2	Additional backup API	Third candidate if configured
REACT_APP_SOCKET_URL	Optional socket base URL	Overrides socket URL derivation

Appendix E - Recommended Evidence Screenshots

1. Landing/home page
2. Registration + validation
3. Google/GitHub authentication
4. Email verification / 2FA
5. Dashboard
6. Mentor discovery and request
7. Job board + application
8. Resume analyzer with structured result
9. AI career chat
10. Mock interview + evaluation
11. Forum moderation / posting
12. Real-time chat + presence
13. Notifications
14. Developer activity dashboard
15. Leaderboard
16. Admin dashboard
17. Android APK running in emulator/device
18. GitHub Actions successful test run
19. GitHub Actions APK artifact/release
20. Network failure / fallback demonstration (where safe to reproduce)

Appendix F - Submission Note

Before final academic submission, replace the cover placeholders for institution, guide and register number, add selected runtime screenshots, and update any statements about live hosting or external-service configuration using the actual deployment state on the submission date. Do not expose API keys, private credentials or personal tokens in screenshots or appendices.